

Leetcode 203 : remove list elements.

```

struct ListNode* removeElements(struct ListNode* head, int val) {
    struct ListNode *temp1 = head, *temp2 = head, *prev = NULL;
    while (temp1 != NULL) {
        temp2 = temp1;
        while (temp2 != NULL && temp2->val != val) {
            prev = temp2;
            temp2 = temp2->next;
        }
        if (temp2 == NULL) break;
        if (temp2 == head) {
            head = head->next;
            free(temp2);
            temp1 = head;
            prev = NULL;
        } else {
            prev->next = temp2->next;
            free(temp2);
            temp1 = prev->next;
        }
    }
    return head;
}

```

O/P
 head = [1, 2, 6, 3, 4, 5, 6]
 val = 6
 output:
 [1, 2, 3, 4, 5]

Leetcode 876 (Middle of Linked list)

```

struct ListNode* middleNode(struct ListNode* head) {
    struct ListNode *temp = head;
    int count = 1;
    while (temp->next != NULL) {
        temp = temp->next;
        count++;
    }
    if (count % 2 == 0) {
        struct ListNode *temp = head;
        for (int i = 1; i < count/2; i++) {
            temp = temp->next;
        }
        return temp->next;
    } else {
        if (count == 1) {
            return temp;
        }
        struct ListNode *temp = head;
        for (int i = 1; i < count/2; i++) {

```

```

temp = temp->next;
}
return temp->next;
}
}

```

Output:

Input:

head = [1, 2, 3, 4, 5, 6]

Output: [4, 5, 6]

Leetcode 141: Linked List cycle.

```

bool hasCycle ( struct ListNode *head ) {
    if ( head == NULL || head->next == NULL ) {
        return false;
    }
    struct ListNode *slow = head;
    struct ListNode *fast = head;
    while ( fast != NULL && fast->next != NULL ) {
        slow = slow->next;
        fast = fast->next->next;
        if ( slow == fast ) {
            return true;
        }
    }
    return false;
}

```

Input:

head = [3, 2, 0, -4]

pos = 1

Output:

true.

leetcode 617 : Merge two Binary trees

```
struct TreeNode* mergeTrees(struct TreeNode* root1, struct
                             TreeNode* root2) {
    if (root1 == NULL) {
        return root2;
    }
    if (root2 == NULL) {
        return root1;
    }
    root1->val = root1->val + root2->val;
    root1->left = mergeTrees(root1->left, root2->left);
    root1->right = mergeTrees(root1->right, root2->right);
    return root1;
}
```

input :

root1 = [1, 3, 2, 5]

root2 = [2, 1, 3, null, 4, null, 7]

output :

[3, 4, 5, 5, 4, null, 7]

Handwritten note:
Pi - 28/12/21